

软件工程常用术语整理

面向 Java 后端、架构设计、AI Agent 开发学习。

1. 需求阶段

PRD (Product Requirement Document)

产品需求文档

描述产品要做什么。

关注：

- 用户是谁
- 解决什么问题
- 有哪些功能
- 产品目标是什么

示例：

功能：

- 用户注册
- 用户登录
- 用户聊天

目标：

提升用户交流效率

主要编写者：

- 产品经理
 - 项目负责人
-

BRD (Business Requirement Document)

商业需求文档

关注：

为什么要做这个产品？

内容：

- 商业价值
- 市场机会
- 收益目标

MRD（Market Requirement Document）

市场需求文档

关注：

市场为什么需要这个产品？

例如：

用户需要低成本即时通信工具
传统短信成本较高

SRS（Software Requirement Specification）

软件需求规格说明书

比 PRD 更技术化。

描述：

- 功能需求
- 非功能需求
- 性能要求
- 接口要求

例如：

FR-001：

用户可以发送消息

限制：

消息最大长度1000字符

响应：

1秒内返回

关系：

BRD
↓
MRD
↓
PRD
↓
SRS
↓
代码实现

2. 软件设计模型

UML (Unified Modeling Language)

统一建模语言。

用于描述系统设计。

常见：

类图

描述：

- 类
- 属性
- 方法
- 关系

时序图

描述：

对象之间调用过程。

例如：

用户
↓
Controller
↓

Service



Database

状态图

描述状态变化。

例如：

订单创建



支付成功



发货



完成

3. 软件架构模式

MVC（Model View Controller）

模型-视图-控制器。

核心：

分离：

- 数据
- 页面
- 请求处理

结构：

User



Controller



Model



View

Web 后端常见扩展：

Controller



Service



DAO



Database

适合：

- Web 应用
- 管理后台
- CRUD 系统

MVVM（Model View ViewModel）

模型-视图-视图模型。

常用于：

- Vue
- Angular
- Android

特点：

数据自动绑定。

MVP (Model View Presenter)

MVC 的变体。

常用于：

- Android
- 桌面应用

单体架构 (Monolith)

所有功能在一个应用中。

例如：

Application

|— User
|— Order
|— Payment
|— Inventory

优点：

- 简单
- 易开发

缺点：

- 大规模后难维护

微服务架构 (Microservices)

将系统拆分为多个独立服务。

例如：

User Service

Order Service

Payment Service

Inventory Service

特点：

- 独立部署
- 独立扩展

适合：

大型系统。

SOA（Service Oriented Architecture）

面向服务架构。

微服务的前身。

特点：

通过服务进行系统组合。

Clean Architecture（整洁架构）

核心：

业务逻辑不依赖具体技术。

结构：

UI

↓

Application

↓

Domain

↓

Infrastructure

例如：

业务代码不依赖：

- MySQL
- Redis
- Spring

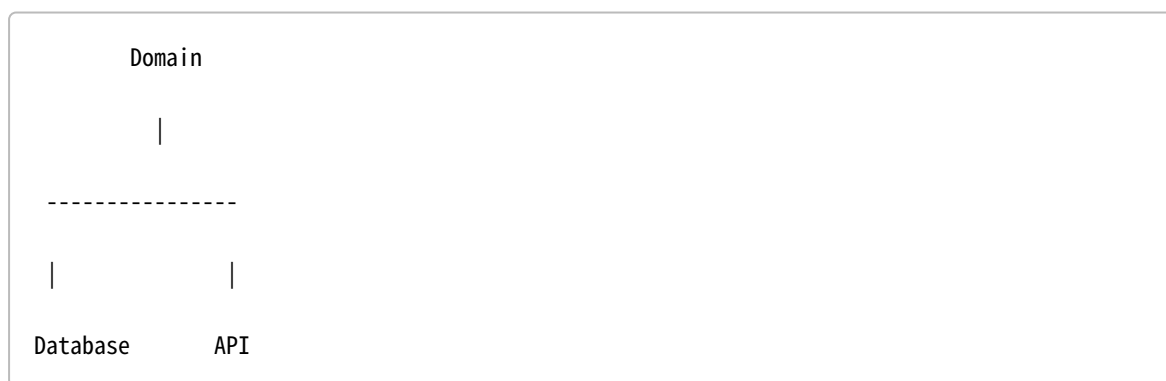
Hexagonal Architecture（六边形架构）

又称：

Ports and Adapters。

核心：

业务作为中心。



适合：

- 微服务
- 企业系统

4. 领域设计

DDD（Domain Driven Design）

领域驱动设计。

核心：

根据业务领域设计软件。

传统：

User表
Order表
Product表

DDD:

Customer

Order

Payment

Inventory

主要概念:

Entity (实体)

具有唯一身份。

例如:

User(id=100)

Value Object (值对象)

描述属性，没有唯一身份。

例如:

Money

Address

Date

Aggregate (聚合)

业务一致性边界。

例如:

Order

└── OrderItem

└── Payment

Domain Event（领域事件）

描述业务发生。

例如：

OrderCreated

PaymentCompleted

适合：

- 电商
- 金融
- ERP
- SaaS

5. 开发方法

TDD（Test Driven Development）

测试驱动开发。

流程：

写测试

↓

测试失败

↓

实现代码

↓

测试通过



重构

适合：

- 金融
- 密码系统
- 核心业务

BDD（Behavior Driven Development）

行为驱动开发。

关注用户行为。

格式：

Given:

用户登录页面

When:

输入正确密码

Then:

登录成功

ATDD（Acceptance Test Driven Development）

验收测试驱动开发。

关注：

客户需求是否满足。

Specification Driven Development

规范驱动开发。

核心：

先定义规格，再开发。

流程：

需求

↓

Specification

↓

Implementation

↓

Test

AI Coding 场景非常重要。

6. 测试类型

Unit Test（单元测试）

测试单个函数。

例如：

```
calculatePrice()
```

Integration Test（集成测试）

测试多个模块协作。

例如：

Controller



Service



Database

E2E Test (End To End)

端到端测试。

模拟真实用户：

打开网页

登录

购买

支付

7. 数据访问

ORM (Object Relational Mapping)

对象关系映射。

将：

Java对象

映射到：

数据库表。

例如：

User对象

↓

user_table

常见：

- Hibernate
- MyBatis

DAO（Data Access Object）

数据访问对象。

负责：

数据库操作。

例如：

UserDAO

save()

findById()

Repository

DDD中的数据访问抽象。

区别：

DAO：

关注数据库。

Repository：

关注领域对象。

8. 分布式架构

Event Driven Architecture（事件驱动架构）

系统通过事件通信。

例如：

```
graph TD; A[OrderCreated] --> B[Inventory Service]; B --> C[Payment Service];
```

OrderCreated

↓

Inventory Service

↓

Payment Service

适合：

- 微服务
- 多 Agent 系统

Message Queue（消息队列）

用于异步通信。

常见：

- Kafka
- RabbitMQ
- RocketMQ

作用：

- 解耦
 - 削峰
 - 异步处理
-

9. 工程流程

CI/CD

持续集成 / 持续部署。

CI:

提交代码

↓

自动测试

↓

自动构建

CD:

测试通过

↓

自动发布

DevOps

Development + Operations。

开发运维一体化。

包含：

- CI/CD
- Docker
- Kubernetes
- 监控

Git Flow

代码管理流程。

例如：

```
main  
  
develop  
  
feature/login  
  
bugfix/payment
```

10. AI Agent 时代相关概念

Agent Skill

给 AI Agent 的能力模块。

包含：

- 工作流程
- 工具使用方式
- 工程经验

Multi-Agent

多智能体系统。

例如：

```
需求 Agent  
↓  
架构 Agent  
↓  
开发 Agent  
↓  
测试 Agent  
↓
```

Review Agent

Agent Communication Protocol

Agent 通信协议。

用于：

- 消息传递
- 任务分配
- 状态同步

11. 推荐学习组合

小项目

例如：

博客系统：

MVC

+

简单测试

中型项目

例如：

聊天系统：

DDD

+

Clean Architecture

+

TDD

+

Specification

大型企业系统

DDD

+

Clean Architecture

+

Microservices

+

Event Driven

+

TDD

+

CI/CD

AI Agent 开发推荐路线

PRD

↓

Specification

↓

DDD

↓

Architecture Design

↓

Coding Agent

↓

Testing Agent

↓

Deployment Agent

未来的软件开发趋势：

从：

人写代码

转向：

人设计目标和约束

↓

Agent 完成工程流程